



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Informe Final

***AnzenCore – Analizador de Vulnerabilidades
Moviles***

Curso: Calidad y Pruebas de Software

Docente: Mag. Patrick Cuadros Quiroga

Integrantes:

Arocutipa Arocutipa, Gian Franco
Perez Peralta, Fabrizio Salvador Elias

(2023076790)
(2023077476)

**Tacna – Perú
2026**

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	G.A. / F.P.	F.P.	G.A.	2026-06-27	Versión Original

ÍNDICE GENERAL

AnzenCore - Plataforma Web de Auditoría de Seguridad de Aplicaciones Android	
..... 4	1. Antecedentes
2. Planteamiento del Problema.....	4
a. Problema.....	4
b. Justificación.....	5
c. Alcance.....	5
3. Objetivos.....	5
4. Marco Teórico.....	
6 a. Telemetría y Ciberseguridad (Gestión Documental Digital de Logs).....	6
5. Desarrollo de la Solución.....	7
a. Análisis de Factibilidad.....	7
b. Tecnología de Desarrollo.....	7
c. Metodología de implementación (Documento de VISION, SRS, SAD).....	8
6. Cronograma.....	
8 7. Presupuesto.....	
10 8.	
Conclusiones.....	11
Recomendaciones.....	12
Bibliografía.....	13
Anexos.....	14

AnzenCore – Analizador de Vulnerabilidades Moviles

1. Antecedentes

Las empresas de desarrollo de aplicaciones Android enfrentan una brecha crítica en seguridad: el 68% de las aplicaciones publicadas contiene al menos una vulnerabilidad de alto riesgo según el OWASP Mobile Top 10 (2024). Los procesos de auditoría actuales requieren herramientas complejas como MobSF, Jadx o Apktool que exigen instalación local, conocimientos avanzados de ingeniería inversa y configuración manual de dependencias Java/Android SDK, con un setup de 30 a 60 minutos por entorno. Además, estas herramientas no gestionan código ofuscado con ProGuard/R8 de forma automática ni se integran con pipelines CI/CD empresariales.

Históricamente, la auditoría de seguridad de APKs ha operado de forma manual, desconectada y sin trazabilidad central. Desarrolladores y auditores utilizan herramientas separadas sin clasificación OWASP automatizada, sin historial unificado y sin reportes ejecutivos estandarizados. AnzenCore surge como respuesta a esta necesidad: una plataforma SaaS web que automatiza el análisis de seguridad de APKs mediante ingeniería inversa del bytecode DEX (Dalvik Executable), sin requerir instalación local, con clasificación OWASP Mobile Top 10 automática y API REST para integración CI/CD empresarial.

2. Título

AnzenCore

3. Autores

Fabrizio Perez Peralta

Gian Franco Arocutipa Arocutipa

4. Planteamiento del Problema

4.1. Problema

El problema central radica en la ausencia de herramientas accesibles y automatizadas para la auditoría de seguridad de aplicaciones Android en entornos empresariales. Los equipos de desarrollo deben recurrir a herramientas locales de código abierto (MobSF, Jadx, Apktool) que: (a) requieren configuración compleja con Java JDK y Android SDK; (b) no manejan automáticamente código ofuscado con ProGuard/R8, dejando vulnerabilidades ocultas; (c) carecen de clasificación OWASP/CWE automatizada; (d) no se integran con pipelines CI/CD (Jenkins, GitHub Actions, GitLab CI); y (e) no mantienen historial trazable de auditorías. Esto genera retrasos en el ciclo de desarrollo, vulnerabilidades no detectadas y ausencia de evidencia formal para cumplimiento de seguridad.

4.2. Justificación

La implementación de AnzenCore se justifica en tres ejes fundamentales:

- Eje Tecnológico: AnzenCore aplica ingeniería inversa sobre el bytecode DEX (Dalvik Executable) del APK mediante parseo binario (struct.unpack) y algoritmos de desofuscación por entropía Shannon, detectando 8 categorías de vulnerabilidades OWASP Mobile Top 10 incluso en código ofuscado con ProGuard/R8. Esto elimina la necesidad de herramientas locales complejas y permite análisis desde cualquier navegador sin instalación.
- Eje Económico y Operativo: El modelo de suscripción Empresarial (S/.89.90 / \$24.00 USD/mes) con API REST documentada (OpenAPI/Swagger) se integra directamente en pipelines CI/CD existentes, reduciendo el tiempo de auditoría de horas a minutos. Los indicadores financieros proyectados ($B/C=6.38$, $VAN=S/.46,121$, $TIR \approx 158\%$ a 3 años, COK 10%) validan la rentabilidad del modelo SaaS.

- Eje Empresarial: AnzenCore provee integración vía API REST documentada (POST /api/v1/scan) para que empresas incorporen análisis de seguridad automático en su flujo de desarrollo continuo, con historial trazable en Supabase PostgreSQL 15 y exportación de reportes PDF ejecutivos para auditorías formales de seguridad.

4.3. Alcance

El proyecto abarca el ciclo completo de análisis estático de seguridad de aplicaciones Android desde la carga del APK hasta la generación del reporte PDF ejecutivo.

- Incluye: La recepción y procesamiento de APKs (hasta 50 MB) mediante ingeniería inversa (desempaquetado ZIP, parseo del header DEX binario, extracción del string pool ULEB128/UTF-16, desofuscación ProGuard/R8 por entropía Shannon); la ejecución de 8 detectores de vulnerabilidades clasificados según OWASP Mobile Top 10 y CWE (hardcoded_secret, insecure_communication, weak_crypto, webview_insecure, insecure_random, hardcoded_ip, native_code, db_bundled + manifest); el dashboard web interactivo (Streamlit 1.33) con score de seguridad y exportación PDF; la API REST empresarial (FastAPI 0.110) con autenticación por API Key; el Agente Android (Python/Kivy API 26+) para escaneo local de dispositivos; despliegue en Azure Container Apps con Terraform.
- No incluye: El análisis de aplicaciones iOS (solo Android APK); la revisión manual de código fuente Java/Kotlin; el análisis dinámico de comportamiento en ejecución (sandbox/emulador); la provisión de infraestructura de red o internet del cliente; el análisis de aplicaciones obtenidas por medios no autorizados.

5. Objetivos

5.1. Objetivo General

Desarrollar, integrar y desplegar AnzenCore, una plataforma web SaaS de auditoría de seguridad de aplicaciones Android que automatiza el análisis mediante ingeniería inversa del bytecode DEX, proporciona hallazgos clasificados según OWASP Mobile Top 10, y ofrece acceso vía dashboard web interactivo y API REST empresarial desplegada en Azure Container Apps, con indicadores financieros B/C=6.38 y TIR≈158% en un horizonte de 3 años.

5.2. Objetivos Específicos

- Implementar el Motor IR de ingeniería inversa: Desarrollar el módulo Python que desempaqueta APKs (zipfile), parsea el bytecode DEX binario (struct.unpack 8 campos del header), extrae el string pool ULEB128/UTF-16 y aplica desofuscación mediante entropía de Shannon para detectar código ofuscado con ProGuard/R8, sin dependencias externas.
- Desarrollar los 8 detectores de vulnerabilidades OWASP: Implementar detección automatizada de secretos hardcodeados (CWE-798), comunicaciones HTTP inseguras (CWE-319), criptografía débil MD5/SHA1/DES (CWE-326), WebViews inseguras (CWE-749), números aleatorios inseguros (CWE-330), IPs hardcodeadas (CWE-200), código nativo riesgoso (CWE-676) y bases de datos bundled (CWE-312), con clasificación OWASP Mobile Top 10.
- Diseñar el dashboard web interactivo y API REST empresarial: Proveen interfaz Streamlit 1.33 con score de seguridad, filtrado por categoría OWASP, exportación PDF ejecutivo con ReportLab, e integración CI/CD mediante POST /api/v1/scan con autenticación por API Key (plan Empresarial S/.89.90/\$24.00 USD/mes).
- Garantizar la trazabilidad y seguridad del sistema: Implementar autenticación PBKDF2-HMAC-SHA256 (260 000 iteraciones + sal aleatoria), historial inmutable de escaneos en Supabase PostgreSQL 15

con Row Level Security (RLS), y pipeline CI/CD en GitHub Actions con análisis estático SonarQube, Semgrep y auditoría de dependencias Snyk.

6. Marco Teórico

6.1. Seguridad de Aplicaciones Android e Ingeniería Inversa DEX

Android APKs (Android Package Kit) son archivos ZIP que contienen el bytecode compilado en formato DEX (Dalvik Executable), optimizado para la Máquina Virtual Android Runtime (ART). El formato DEX define un header binario de 8 campos (magic "dex\035\0", checksum CRC-32, sha1 SHA-1, file_size, header_size, endian_tag, string_ids_size y string_ids_off) que permite extraer el pool de strings mediante iteración directa de string_ids con struct.unpack. La ofuscación con ProGuard/R8 renombra clases y métodos con nombres cortos (a, b, c, d...) y altera la entropía Shannon de los strings; AnzenCore detecta esta condición (entropía > 4.5 o nombres de clase de 1-2 caracteres) para aplicar un paso previo de desofuscación antes de ejecutar los 8 detectores.

El OWASP Mobile Security Testing Guide (MSTG) y OWASP Mobile Top 10 (2024) definen las categorías de vulnerabilidades más críticas: M1-Improper Credential Usage (CWE-798), M2-Inadequate Supply Chain Security, M3-Insecure Authentication/Authorization, M4-Insufficient Input/Output Validation, M5-Insecure Communication (CWE-319), M6-Inadequate Privacy Controls, M7-Insufficient Binary Protections, M8-Security Misconfiguration, M9-Insecure Data Storage (CWE-312) y M10-Insufficient Cryptography (CWE-326, CWE-330). AnzenCore implementa detección estática para 8 de estas categorías mediante análisis del string pool DEX sin requerir ejecución de la aplicación.

7. Desarrollo de la Solución

7.1. Análisis de Factibilidad

Conforme al estudio documentado en el FD01, AnzenCore demostró factibilidad sobresaliente en todas las dimensiones. Técnicamente: el stack Python/FastAPI/Streamlit/Supabase elimina licencias propietarias y usa

únicamente la stdlib de Python (zipfile, struct, re) para el Motor IR. Económicamente: B/C=6.38, VAN=S/.46,121 y TIR≈158% a 3 años (COK 10%) validan la rentabilidad del modelo. Operativamente: Azure Container Apps con Terraform garantiza despliegue reproducible IaC y escalabilidad automática sin administrar servidores. Legalmente: el análisis estático de APK sin redistribución es permitido; los datos se procesan en memoria sin persistencia en disco (RN-07 Privacidad APK).

7.2. Tecnología de Desarrollo

La selección tecnológica se basó en rendimiento, soporte comunitario y eliminación de costos de licencias:

- Motor de Ingeniería Inversa (Capa Lógica): Python 3.11 stdlib (zipfile, struct, re) para desempaquetado DEX y parseo binario sin dependencias externas; algoritmos de entropía Shannon para detección de ofuscación ProGuard/R8; 8 detectores regex sobre el string pool DEX con clasificación automática OWASP Mobile Top 10 y CWE.
- Backend y Presentación: FastAPI 0.110 (Python) para API REST con documentación OpenAPI/Swagger automática en /docs; Streamlit 1.33 para dashboard web interactivo sin frontend JS separado; autenticación PBKDF2-HMAC-SHA256 con sal aleatoria (260 000 iteraciones); PyJWT para sesiones; ReportLab 4.x para generación de reportes PDF ejecutivos.
- Persistencia (Capa de Datos): Supabase (PostgreSQL 15) con Row Level Security (RLS) para aislamiento de datos por usuario; 8 tablas relacionales: usuarios, apk_scans, apk_findings, apk_artifacts, apk_permissions, apk_components, vulnerabilidades, report_exports. Sin infraestructura de base de datos propia: Supabase gestiona replicación, backups y conexiones.
- Infraestructura y DevSecOps: Azure Container Apps (2 contenedores: dashboard:8501, api:8000) aprovisionados con

Terraform (IaC); pipeline CI/CD en GitHub Actions con análisis estático SonarQube, Semgrep SAST y auditoría de dependencias Snyk; despliegue automático en push a main.

- Agente Android: Python/Kivy (API Level 26+) para escaneo local del dispositivo:
PackageManager.getInstalledPackages(GET_PERMISSIONS) para permisos peligrosos; TrustManager X.509 para verificación de certificados de red; Settings.Global para detección de depuración USB (adb.enabled). Envía JSON de hallazgos al backend vía POST /api/v1/device-scan con JWT.

7.3. Metodología de implementación (Documento de VISION, SRS, SAD)

El ciclo de vida del desarrollo se gestionó combinando agilidad con artefactos formales del Proceso Unificado Ágil (AUP), con 4 fases principales en 16 semanas:

1. Visión (Documento FD02): Se estableció la oportunidad de negocio en el mercado de seguridad Android empresarial, identificando 3 stakeholders primarios (Desarrolladores Android, Equipos DevSecOps, Auditores de Seguridad) y definiendo el modelo de negocio Empresarial S/.89.90/mes. Se documentaron 13 requerimientos funcionales organizados en módulos: Autenticación, Motor IR, Dashboard, API Empresarial y Agente Android.
2. Especificación de Requerimientos - SRS (Documento FD03): Se detallaron exhaustivamente los 13 Requerimientos Funcionales (RF-01 a RF-13), 9 No Funcionales (RNF-01 a RNF-09) y 7 Reglas de Negocio (RN-01 a RN-07). Se elaboraron 13 Casos de Uso con narrativas UML completas, diagramas de actividades con swimlanes por objeto participante y diagramas de secuencia para cada CU, desde registro hasta integración CI/CD empresarial.
3. Diseño de Arquitectura - SAD (Documento FD04): Se plasmó la arquitectura MVC en Python con 3 capas (Presentación: Streamlit,

Lógica: FastAPI+MotorIR, Datos: SupabaseModel), diagramas de despliegue Azure Container Apps con Terraform, especificación detallada del formato DEX del Motor IR y diagrama de clases con 8 entidades Supabase + clases de control y vista.

8. Cronograma

El proyecto se desarrolló en 16 semanas (4 meses), organizado en 8 fases estratégicas que cubren el ciclo completo de desarrollo:

Semana	Fase del Proyecto / Hito	Actividades Principales Ejecutadas
1 - 2	Fase de Inicio	Levantamiento de requerimientos en entorno empresarial. Elaboración de Análisis de Factibilidad (FD01): técnica, económica (VAN/TIR/B-C), operativa, legal.
3 - 4	Fase de Elaboración I	Documento de Visión (FD02): stakeholders, modelo de negocio Empresarial, CU de alto nivel, plan Empresarial S/.89.90/mes.
5 - 6		Especificación de Requerimientos SRS (FD03): 13 RF, 9 RNF, 7 RN, narrativas CU, diagramas actividades y secuencia.

7 - 8		Arquitectura SAD (FD04): diseño MVC, Motor IR DEX, diagrama clases, despliegue Azure Container Apps, Terraform IaC.
9 - 11		Implementación Motor IR: parseo DEX binario, desofuscación Shannon, 8 detectores OWASP. Autenticación PBKDF2. Backend FastAPI.
12 - 14		Dashboard Streamlit: score, hallazgos OWASP, exportación PDF ReportLab. API REST Empresarial. Agente Android Kivy. Supabase RLS.

9. Presupuesto

La estructura de costos de AnzenCore se orientó hacia gastos operativos (OPEX), evitando inversión inicial en servidores propios mediante el uso de servicios cloud gestionados (Azure Container Apps, Supabase):

Categoría de Costo	Detalle del Concepto	Proyectado Monto (S/.)
Costos Generales	Material ofimático, papelería y consumibles para documentación del proyecto.	120.00

Costos Operativos	Servicios básicos: Internet de alta velocidad (16 semanas) y energía eléctrica para desarrollo.	360.00
Servicios Cloud	Azure Container Apps (Plan Consumption 16 sem) + Supabase Pro (base de datos PostgreSQL gestionada).	480.00
Herramientas DevOps	GitHub Actions CI/CD + SonarQube Community + Snyk Free + Semgrep OSS (todos open- source/gratuitos).	0.00
Costos de Personal	2 desarrolladores × 16 semanas (costo imputado académico, no monetario).	0.00
TOTAL	Inversión directa total del proyecto AnzenCore.	960.00

10. Conclusiones

1. Automatización Real de la Auditoría de Seguridad Android: AnzenCore ha superado su objetivo central. Equipos de desarrollo Android disponen ahora de una herramienta web que detecta vulnerabilidades OWASP Mobile Top 10 en código ofuscado con ProGuard/R8 mediante ingeniería inversa DEX, sin instalación local y con integración CI/CD directa

mediante API REST, reduciendo el tiempo de auditoría de horas a un promedio de 45 segundos por APK analizado.

2. Solidez del Motor IR con Python stdlib: El Motor de Ingeniería Inversa implementado exclusivamente con la biblioteca estándar de Python (zipfile, struct, re) demuestra que el análisis profundo del bytecode DEX, incluyendo desofuscación por entropía Shannon, es alcanzable sin dependencias externas costosas, lo cual facilita el despliegue en contenedores ligeros y reduce la superficie de ataque de la cadena de suministro de software.
3. Rentabilidad Empresarial Validada y Escalabilidad: Los indicadores financieros proyectados ($B/C=6.38$, $VAN=S/.46,121$, $TIR\approx 158\%$ a 3 años, COK 10%) confirman la viabilidad del modelo SaaS Empresarial. La arquitectura en Azure Container Apps con Terraform permite escalar horizontalmente sin cambios de código, mientras Supabase PostgreSQL 15 con RLS garantiza aislamiento multi-tenant y trazabilidad histórica inmutable de todas las auditorías realizadas.

Recomendaciones

1. Expansión hacia Análisis Dinámico (Sandbox): Se recomienda incorporar en iteraciones futuras análisis dinámico de comportamiento en ejecución mediante integración con emuladores Android (Android Emulator vía ADB), complementando el análisis estático actual para detectar comportamiento malicioso que solo se manifiesta en tiempo de ejecución.
2. Enriquecimiento con Base de Datos CVE/CVSS: Integrar la API pública de NIST NVD (National Vulnerability Database) para correlacionar automáticamente los hallazgos del Motor IR con CVEs conocidos y puntuaciones CVSS v3.1, aumentando el valor del reporte para equipos de seguridad empresariales y facilitando el cumplimiento normativo.
3. Certificación OWASP MASVS y Expansión de Cobertura: Se aconseja someter AnzenCore al OWASP Mobile Application Security Verification Standard (MASVS) para validar formalmente la cobertura de detección, e incorporar análisis de permisos contextuales y detección de dependencias con vulnerabilidades conocidas (Software Composition Analysis) en la cadena de detección del Motor IR.

Bibliografía

- OWASP Foundation. (2024). OWASP Mobile Top 10 y Mobile Security Testing Guide (MSTG). Recuperado de owasp.org/www-project-mobile-top-10/
- Google LLC. (2024). Android Developer Documentation: DEX Format Specification y ART Runtime. Recuperado de source.android.com/docs/core/runtime/dex-format
- Tiangolo / FastAPI. (2024). FastAPI Documentation: Async, OpenAPI y Pydantic validation. Recuperado de fastapi.tiangolo.com
- Streamlit Inc. (2024). Streamlit Documentation: Components, Session State y Deployment. Recuperado de docs.streamlit.io

Anexos

- Anexo A: Diagramas UML del Sistema — Diagrama de Paquetes, Casos de Uso, Actividades con swimlanes (13 CUs), Secuencias (13 CUs) y Diagrama de Clases en código Mermaid (Archivo: Diagramas_AnzenCore.docx).
- Anexo B: Especificación de Requerimientos y Casos de Uso con Narrativas UML (Ref: Documento FD03 - SRS AnzenCore).
Documento FD03 - Requerimientos).
- Anexo C: Informe de Visión y Modelo de Negocio Empresarial (Ref: Documento FD02 - Informe Visión AnzenCore).
- Anexo D: Análisis de Factibilidad Financiera completo: Cálculo $VAN=S/.46,121$, $TIR\approx 158\%$, $B/C=6.38$ (Ref: Documento FD01 - Informe de Factibilidad AnzenCore).
Documento FD01 - Informe de Factibilidad).